PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Engenharia de Software

Pedro Henrique Maia Alves
Matrícula: 848301

ANÁLISE DE EFICÁCIA DE TESTES COM TESTE DE MUTAÇÃO

Belo Horizonte

2026

1. Análise inicial

Inicialmente, foram executados os 50 testes previamente existentes no projeto, sendo todos concluídos com sucesso. Em seguida, foi gerado o relatório de cobertura de código por meio da ferramenta Jest. Os resultados indicaram uma cobertura de linhas de 98,64% e de funções de 100%, sugerindo, inicialmente, uma abrangência satisfatória dos testes. Entretanto, a cobertura de ramos atingiu apenas 58,82%, evidenciando que determinados caminhos de execução não estavam sendo devidamente exercitados.

Na etapa seguinte, a ferramenta StrykerJS foi configurada e utilizada para a execução dos testes de mutação. Ao todo, foram gerados 213 mutantes, dos quais 154 foram eliminados pelos testes, 3 apresentaram timeout, 44 sobreviveram e 12 não foram cobertos. Com isso, o mutation score inicial registrado foi de 73,71%.

A análise desses resultados evidencia que uma elevada cobertura de código não garante, necessariamente, a eficácia dos testes. Embora determinada função seja integralmente executada durante os testes, a utilização de entradas pouco diversificadas ou de asserções insuficientes pode comprometer a identificação de comportamentos incorretos. Dessa forma, alterações que introduzem defeitos no código podem permanecer sem detecção, mesmo diante de indicadores elevados de cobertura.

Métrica de cobertura	Resultado inicial
Instruções	85,41%
Ramos	58,82%
Funções	100%
Linhas	98,64%

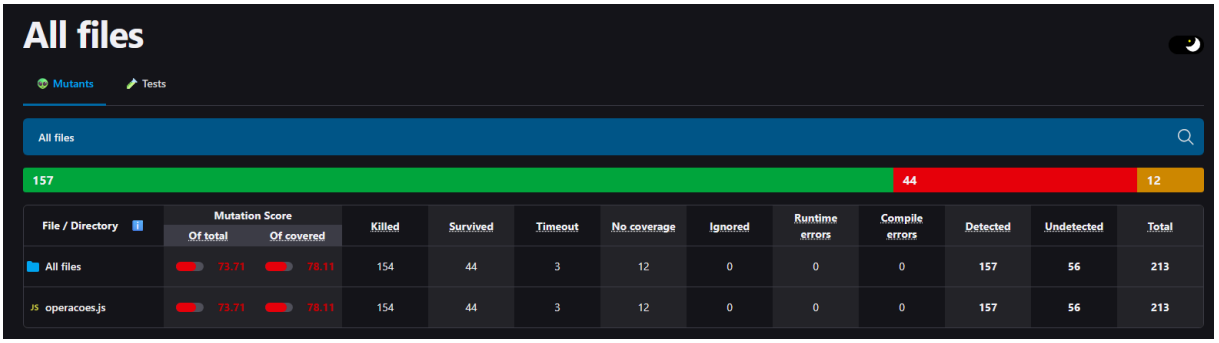


Figura 1 - Resultado inicial do teste de mutação.

2. Análise dos mutantes sobreviventes

2.1 Comparação de valores

No mutante 177, o Stryker trocou > por >= na função isMaiorQue. O teste original usava isMaiorQue(10, 5), então as duas versões retornavam verdadeiro. Percebi que faltava o caso de igualdade.

Solução:

```
expect(isMaiorQue(5, 5)).toBe(false);
```

Com valores iguais, o código original retorna falso e o mutante retorna verdadeiro; por isso o novo teste o mata.

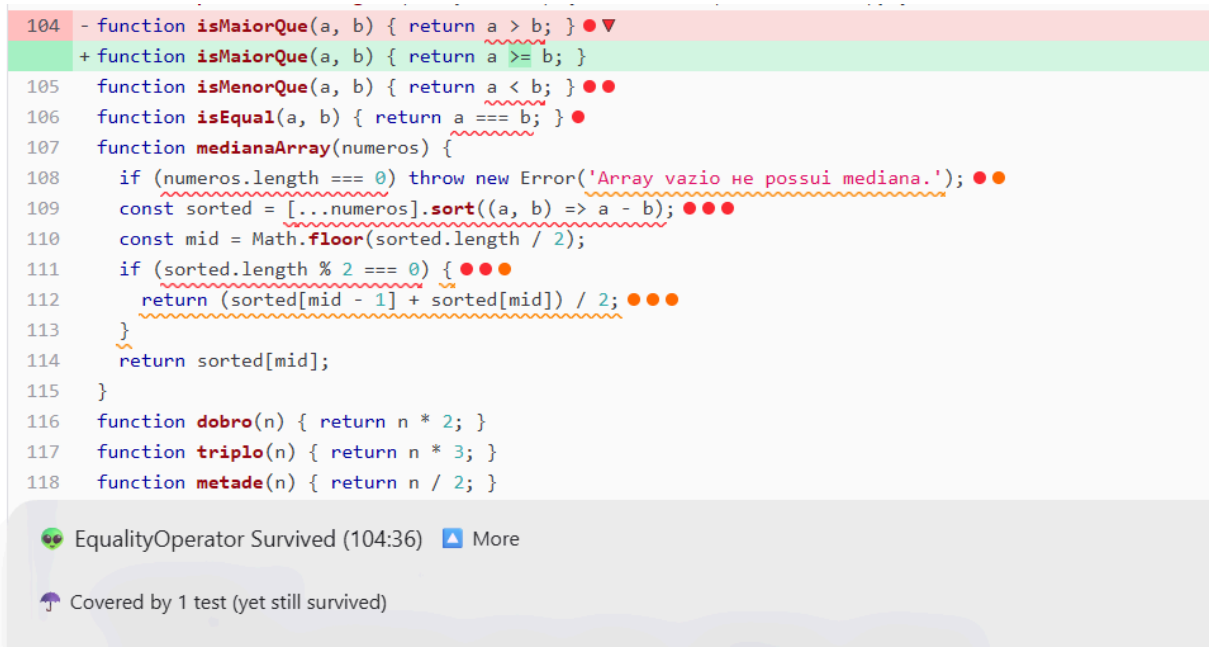


Figura 2 - Mutante 177 (EqualityOperator).

2.2 Conversão de temperatura

Nos mutantes da linha 93, operadores da fórmula foram trocados. O teste original usava somente 0 °C, valor que mascara essas trocas porque qualquer multiplicação iniciada por zero continua sendo zero.

Solução:

```
expect(celsiusParaFahrenheit(100)).toBe(212);
```

Ao usar 100 °C, multiplicação e divisão passam a afetar o resultado. Assim, a alteração do mutante 155 faz o teste falhar.

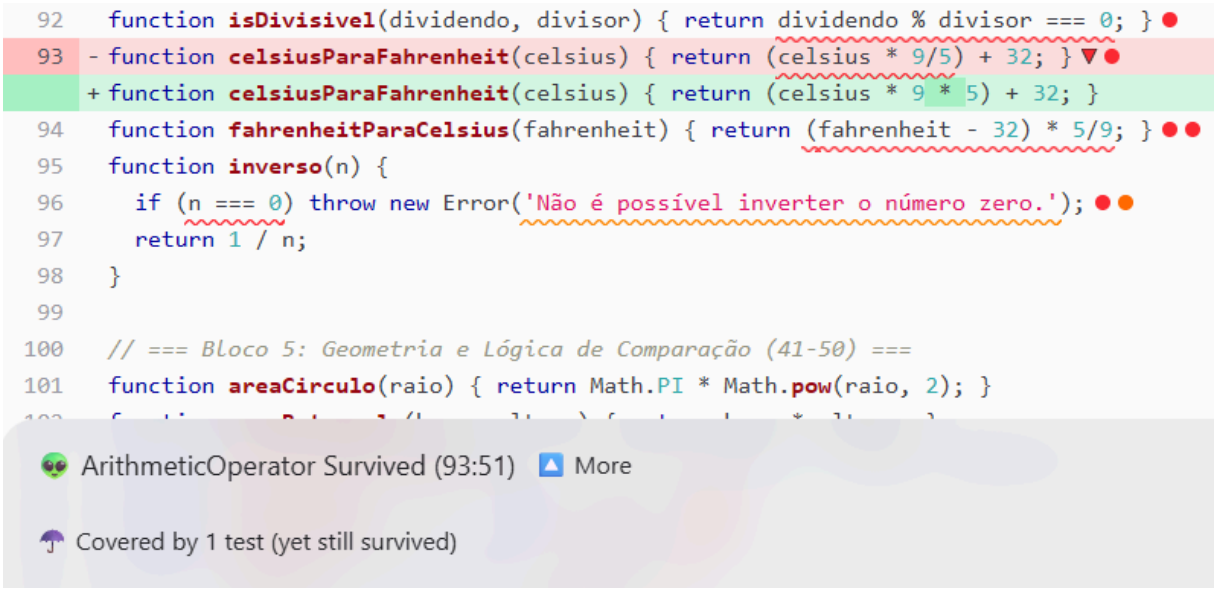


Figura 3 - Mutante 155 (ArithmeticOperator).

2.3 Mediana de um array

O mutante 193 removeu o sort. O teste inicial recebia [1, 2, 3, 4, 5], que já estava ordenado e tinha tamanho ímpar. Desse modo, a remoção não mudava o resultado e o caminho de tamanho par não era executado.

Solução:

```
expect(medianaArray([9, 1, 5])).toBe(5);
expect(medianaArray([10, 2, 4, 8])).toBe(6);
```

O primeiro teste exige a ordenação; o segundo também verifica a média dos dois valores centrais.

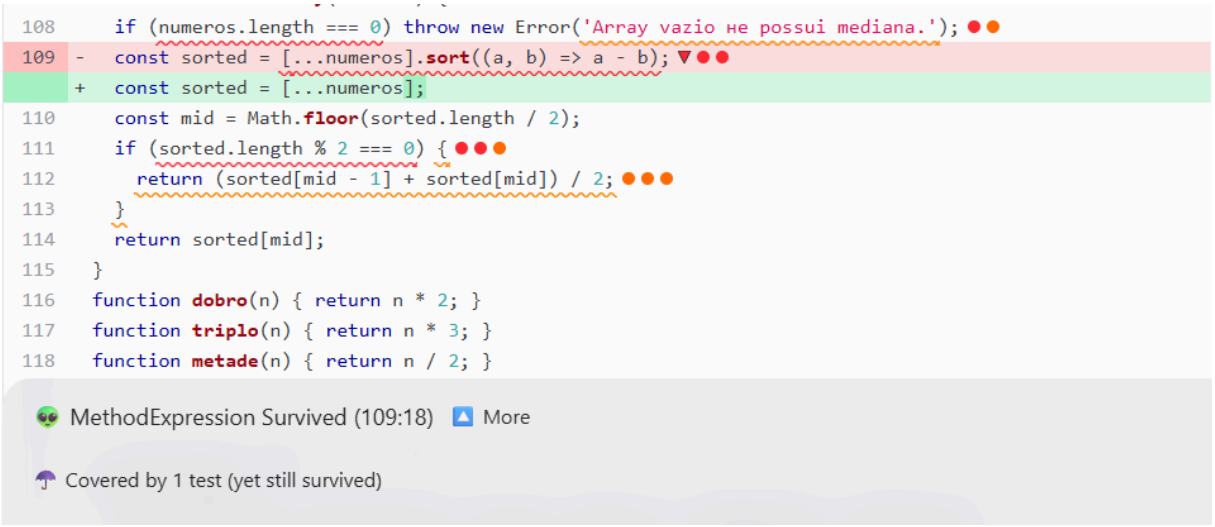


Figura 4 - Mutante 193 (MethodExpression).

3. Resultados finais

Além dos três casos analisados, foram desenvolvidos novos testes contemplando mensagens de erro, arrays vazios, números não primos, resultados booleanos falsos, valores-limite da função clamp e conversão inversa de temperatura. Com essas alterações, a suíte foi ampliada de 50 para 58 testes.

Após a implementação dos novos casos, a ferramenta Jest foi executada novamente, registrando 100% de cobertura em instruções, ramos, funções e linhas. Na primeira execução do StrykerJS após as melhorias, o mutation score atingiu 96,71%, permanecendo sete mutantes equivalentes.

A análise desses mutantes demonstrou que as alterações introduzidas não provocavam diferenças no comportamento observável das funções. Os casos identificados estavam associados a condições redundantes nas operações de fatorial e produto de arrays, além de verificações relacionadas aos limites da função clamp.

Diante disso, os mutantes equivalentes foram documentados por meio de anotações específicas do StrykerJS. Como essas anotações são aplicadas conforme o tipo de mutador presente em determinada linha, o relatório final contabilizou 11 mutantes ignorados.

Na execução final, foram considerados 202 mutantes, dos quais 199 foram eliminados pelos testes e 3 apresentaram timeout. Não foram registrados mutantes sobreviventes ou sem cobertura, resultando em um mutation score final de 100%.

Métrica	Inicial	Final
Testes	50	58
Cobertura de ramos	58,82%	100%
Mutation score	73,71%	100%
Sobreviventes	44	0
Sem cobertura	12	0

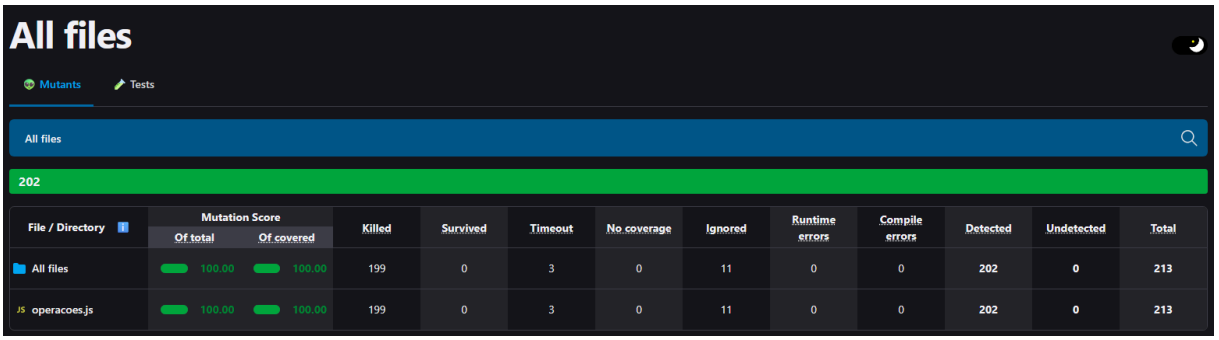


Figura 5 - Resultado final do teste de mutação.

4. Conclusão

Os resultados obtidos demonstram que indicadores elevados de cobertura de código não representam, isoladamente, uma garantia da eficácia dos testes de software. Embora a suíte inicial apresentasse cobertura expressiva de linhas e funções, a aplicação dos testes de mutação revelou limitações relacionadas à diversidade das entradas, à ausência de casos-limite e à insuficiência de determinadas verificações.

A utilização do StrykerJS possibilitou a identificação dessas fragilidades e orientou a elaboração de novos casos de teste, contribuindo para o aprimoramento da suíte. Como resultado, observou-se um aumento da cobertura de ramos de 58,82% para 100%, acompanhado da evolução do mutation score de 73,71% para 100%, após o tratamento dos mutantes equivalentes.

Dessa forma, conclui-se que o teste de mutação constitui uma técnica complementar relevante para a avaliação da qualidade dos testes, pois permite identificar deficiências que não são evidenciadas exclusivamente pelas métricas tradicionais de cobertura. A aplicação dessa abordagem contribuiu para tornar a suíte mais abrangente e eficaz na detecção de alterações incorretas no código.